

LAPORAN TUGAS 3

disusun untuk memenuhi
tugas mata kuliah Pembelajaran Mesin

oleh :

Alvia Zuhra (2208107010003)

Al-Mahfuzh Fadhlur Rohman (2208107010016)

Nurul Uzratun Nashriyyah (2208107010030)

Ganang Setyo Hadi (2208107010052)

Azimah Al-Huda (2208107010069)



**JURUSAN INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA
2025**

DAFTAR ISI

DAFTAR ISI.....	ii
A. PEMAHAMAN DATASET.....	1
B. EKSPLORASI DATA DAN PRAPEMROSESAN.....	8
C. IMPLEMENTASI MODEL.....	10
D. EVALUASI MODEL.....	13
E. ANALISIS HASIL.....	17

A. PEMAHAMAN DATASET

Nama Dataset dan Sumbernya

Dataset yang digunakan adalah Student Academic Performance Dataset yang diperoleh dari platform Kaggle, diunggah oleh pengguna Rabie El Kharoua. Dataset ini pertama kali dipublikasikan pada tahun 2024 dan merupakan dataset sintetis yang dibuat secara eksklusif untuk keperluan edukasi dan penelitian. Berikut adalah link Dataset: [students-performance-dataset](#)

Deskripsi Singkat tentang Dataset

Dataset ini berisi data komprehensif mengenai 2.392 siswa sekolah menengah atas, mencakup informasi demografis, kebiasaan belajar, keterlibatan orang tua, aktivitas ekstrakurikuler, serta performa akademik. Dataset ini sangat signifikan untuk riset pendidikan karena memungkinkan analisis faktor-faktor yang memengaruhi capaian akademik siswa dan pengembangan model prediktif untuk klasifikasi nilai siswa.

Jumlah Data

- Jumlah sampel: 2.392 siswa
- Jumlah fitur: 13 fitur prediktor (Age, Gender, Ethnicity, ParentalEducation, StudyTimeWeekly, Absences, Tutoring, ParentalSupport, Extracurricular, Sports, Music, Volunteering, GPA)
- Label: 1 kolom target (GradeClass) yang mengklasifikasikan nilai siswa ke dalam lima kategori (A, B, C, D, F)

Format Data

Dataset tersedia dalam format CSV (Comma Separated Values), yang merupakan format standar untuk data tabular yang mudah diproses menggunakan berbagai library pemrograman seperti Pandas dalam Python.

Statistik Deskriptif dan Visualisasi Awal

1. Statistik Deskriptif untuk Kolom Numerik

	count	mean	std	min	25%	50%	75%	max	range	variance	skewness	kurtosis
Age	2392.000	16.469	1.124	15.000	15.000	16.000	17.000	18.000	3.000	1.263	0.036	-1.372
StudyTimeWeekly	2392.000	9.772	5.653	0.001	5.043	9.705	14.408	19.978	19.977	31.954	0.050	-1.136
Absences	2392.000	14.541	8.467	0.000	7.000	15.000	22.000	29.000	29.000	71.697	-0.026	-1.176
GPA	2392.000	1.906	0.915	0.000	1.175	1.893	2.622	4.000	4.000	0.838	0.015	-0.863
GradeClass	2392.000	2.984	1.234	0.000	2.000	4.000	4.000	4.000	4.000	1.523	-0.901	-0.417

Berdasarkan hasil statistik deskriptif terhadap variabel numerik dalam dataset, diketahui bahwa rata-rata usia siswa adalah 16,47 tahun dengan rentang usia antara 15 hingga 18 tahun. Rata-rata waktu belajar per minggu adalah sekitar 9,77 jam, dengan penyimpangan standar cukup tinggi yaitu 5,65, menunjukkan variasi yang cukup besar antar siswa. Jumlah ketidakhadiran (absences) memiliki rata-rata 14,54 kali dalam

setahun, dengan nilai maksimum mencapai 29 kali, menandakan adanya siswa dengan tingkat absensi yang tinggi. Nilai rata-rata GPA siswa adalah 1,91 dari skala 4, menunjukkan performa akademik secara umum berada di bawah standar maksimal. Untuk variabel target GradeClass, nilai rata-ratanya adalah 2,98 yang berada mendekati kelas “C” hingga “D”, dengan distribusi data yang condong ke kiri (negatif skewness), menunjukkan bahwa sebagian besar siswa mendapatkan nilai di kisaran rendah hingga sedang. Variabel-variabel ini menunjukkan kecenderungan distribusi yang relatif simetris hingga condong negatif, serta kurtosis yang rendah, menandakan distribusi data yang cenderung mendatar atau menyebar (platykurtic).

2. Deteksi Outlier Dengan Metode IQR

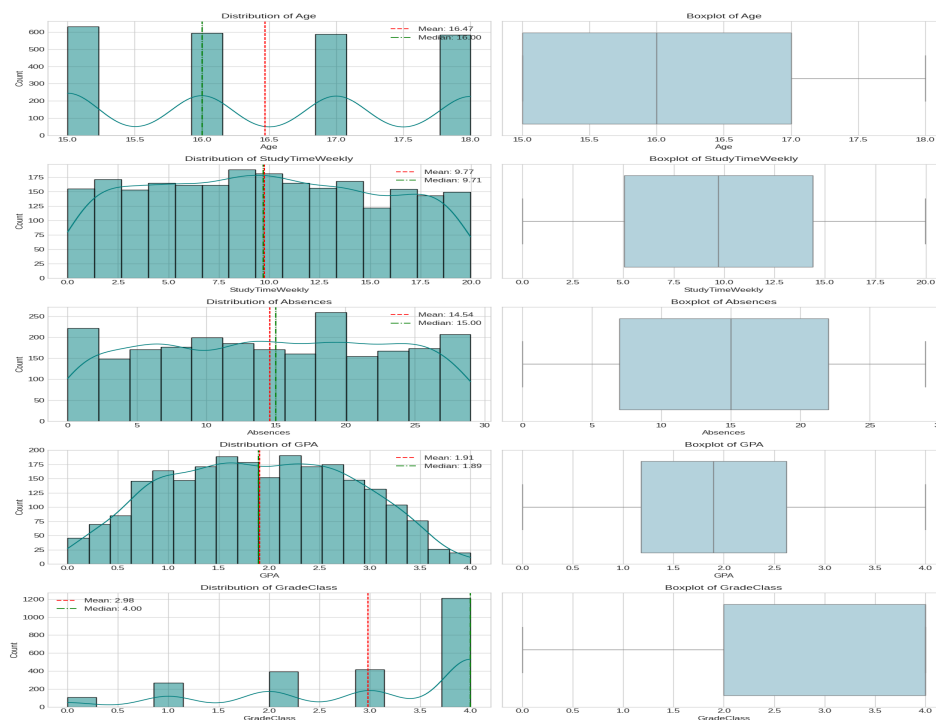
```

=====
OUTLIER DETECTION USING IQR METHOD
=====
Age: 0 outliers detected (0.00% of data)
StudyTimeWeekly: 0 outliers detected (0.00% of data)
Absences: 0 outliers detected (0.00% of data)
GPA: 0 outliers detected (0.00% of data)
GradeClass: 0 outliers detected (0.00% of data)
=====

```

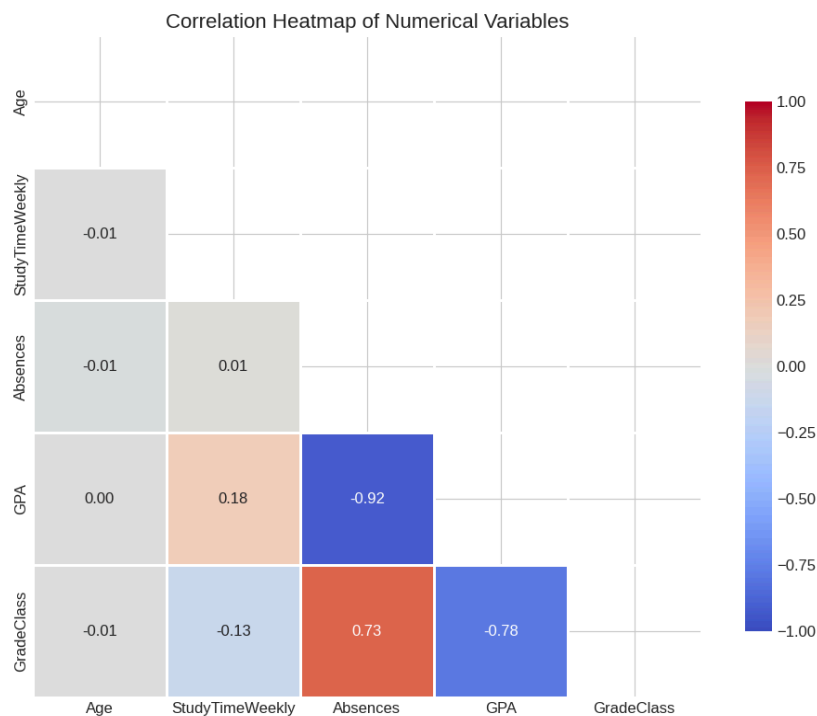
Hasil deteksi outlier menggunakan metode IQR menunjukkan bahwa tidak ada outlier yang terdeteksi pada semua variabel numerik, seperti Age, StudyTimeWeekly, Absences, GPA, dan GradeClass. Ini mengindikasikan bahwa data bersih, terdistribusi secara wajar, dan tidak mengandung nilai ekstrem yang dapat mempengaruhi analisis statistik.

3. Distribusi Variabel Numerik



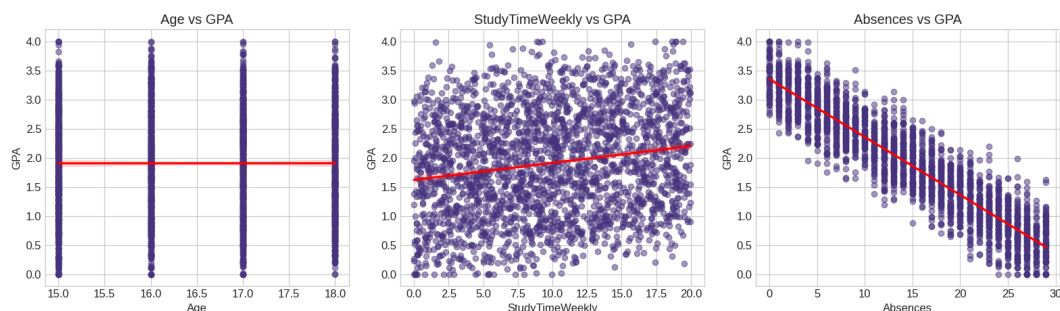
Visualisasi distribusi dan boxplot menunjukkan bahwa usia siswa cenderung merata antara 15 hingga 18 tahun, dengan median di angka 16. Waktu belajar mingguan bervariasi luas, namun mayoritas siswa belajar sekitar 10 jam per minggu. Jumlah absensi berkisar luas dengan sebagian besar siswa memiliki absen di sekitar 15 kali. GPA tersebar relatif merata, namun sebagian besar siswa memiliki nilai di bawah 3. GradeClass paling banyak berada di nilai 4, meskipun reratanya di sekitar 2,98. Secara keseluruhan, data menunjukkan adanya variasi signifikan dalam kebiasaan belajar dan performa akademik.

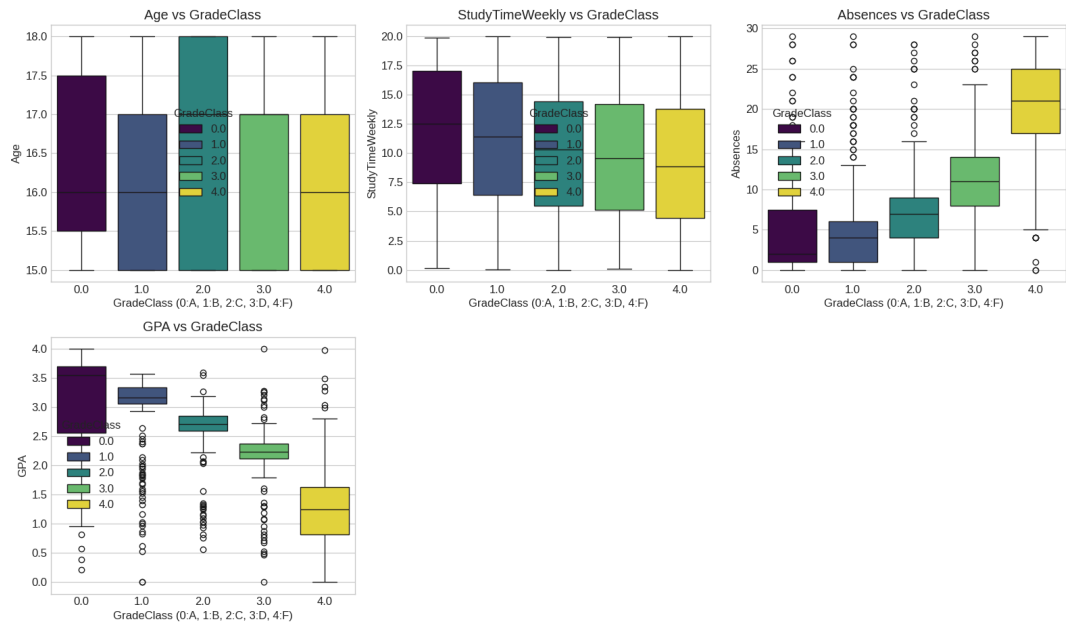
4. Korelasi Antara Variabel Numerik



Berdasarkan heatmap korelasi, terlihat bahwa jumlah absensi memiliki korelasi negatif yang sangat kuat dengan GPA (-0.92) dan cukup kuat dengan GradeClass (-0.78), menandakan bahwa semakin banyak absen, nilai akademik cenderung menurun. Sebaliknya, absensi berkorelasi positif dengan GradeClass (0.73), yang bisa mengindikasikan nilai kelas rendah berkaitan dengan banyaknya absen. Korelasi lainnya relatif lemah dan tidak signifikan, seperti antara usia, waktu belajar, dan variabel lain.

5. Hubungan Antara Fitur Numerik dan Prestasi Akademik





Gambar ini menampilkan hubungan antara variabel numerik seperti usia (Age), waktu belajar mingguan (StudyTimeWeekly), dan jumlah ketidakhadiran (Absences) terhadap nilai akademik siswa, yang diukur melalui GPA dan GradeClass. Terlihat bahwa absensi memiliki korelasi negatif yang kuat terhadap GPA, artinya semakin sering siswa absen, semakin rendah nilai mereka. Sebaliknya, StudyTimeWeekly menunjukkan korelasi positif terhadap GPA meskipun lemah. Visualisasi juga menunjukkan perbedaan distribusi tiap fitur berdasarkan kategori GradeClass, menegaskan bahwa siswa dengan nilai rendah cenderung memiliki absensi lebih tinggi dan GPA lebih rendah.

6. Distribusi Frekuensi Untuk Variabel Kategorikal

Gender Distribution:

	Count	Percentage (%)
0 (Male)	1170	48.910
1 (Female)	1222	51.090

Ethnicity Distribution:

	Count	Percentage (%)
0 (Caucasian)	1207	50.460
1 (African American)	493	20.610
2 (Asian)	470	19.650
3 (Other)	222	9.280

ParentalEducation Distribution:

	Count	Percentage (%)
0 (None)	243	10.160
1 (High School)	728	30.430
2 (Some College)	934	39.050
3 (Bachelor's)	367	15.340
4 (Higher)	120	5.020

Tutoring Distribution:		
	Count	Percentage (%)
0 (No)	1671	69.860
1 (Yes)	721	30.140

ParentalSupport Distribution:		
	Count	Percentage (%)
0 (None)	212	8.860
1 (Low)	489	20.440
2 (Moderate)	740	30.940
3 (High)	697	29.140
4 (Very High)	254	10.620

Extracurricular Distribution:		
	Count	Percentage (%)
0 (No)	1475	61.660
1 (Yes)	917	38.340

Sports Distribution:		
	Count	Percentage (%)
0 (No)	1666	69.650
1 (Yes)	726	30.350

Music Distribution:		
	Count	Percentage (%)
0 (No)	1921	80.310
1 (Yes)	471	19.690

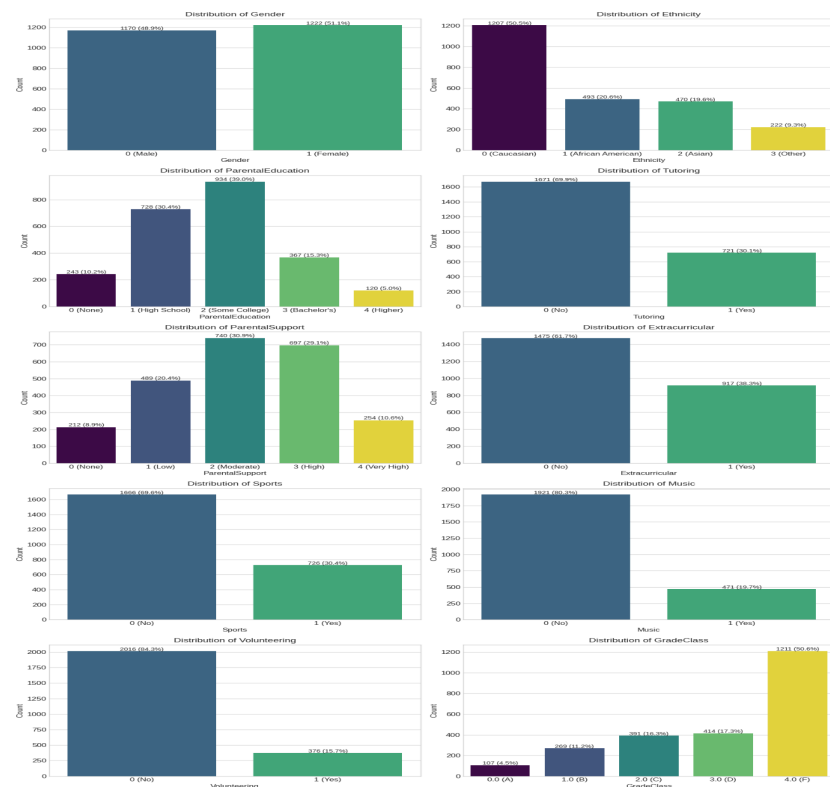
Volunteering Distribution:		
	Count	Percentage (%)
0 (No)	2016	84.280
1 (Yes)	376	15.720

GradeClass Distribution:		
	Count	Percentage (%)
0.0 (A)	107	4.470
1.0 (B)	269	11.250
2.0 (C)	391	16.350
3.0 (D)	414	17.310
4.0 (F)	1211	50.630

Distribusi frekuensi variabel kategorikal menunjukkan bahwa proporsi gender relatif seimbang, dengan perempuan sedikit lebih banyak (51,09%)

dibanding laki-laki (48,91%). Mayoritas siswa berasal dari etnis Kaukasasia (50,46%), diikuti oleh Afrika Amerika (20,61%) dan Asia (19,65%). Sebagian besar orang tua siswa memiliki pendidikan "Some College" (39,05%) atau "High School" (30,43%). Mayoritas siswa tidak menerima les tambahan (69,86%) dan tidak aktif dalam kegiatan ekstrakurikuler (61,66%), olahraga (69,65%), musik (80,31%), maupun kegiatan sukarela (84,28%). Dukungan orang tua paling sering berada pada tingkat "Moderate" (30,94%) dan "High" (29,14%). Namun, distribusi nilai akhir menunjukkan bahwa lebih dari separuh siswa (50,63%) mendapatkan nilai F, menandakan adanya tantangan serius dalam pencapaian akademik yang perlu ditelusuri lebih lanjut.

7. Visualisasi Variabel Kategorikal

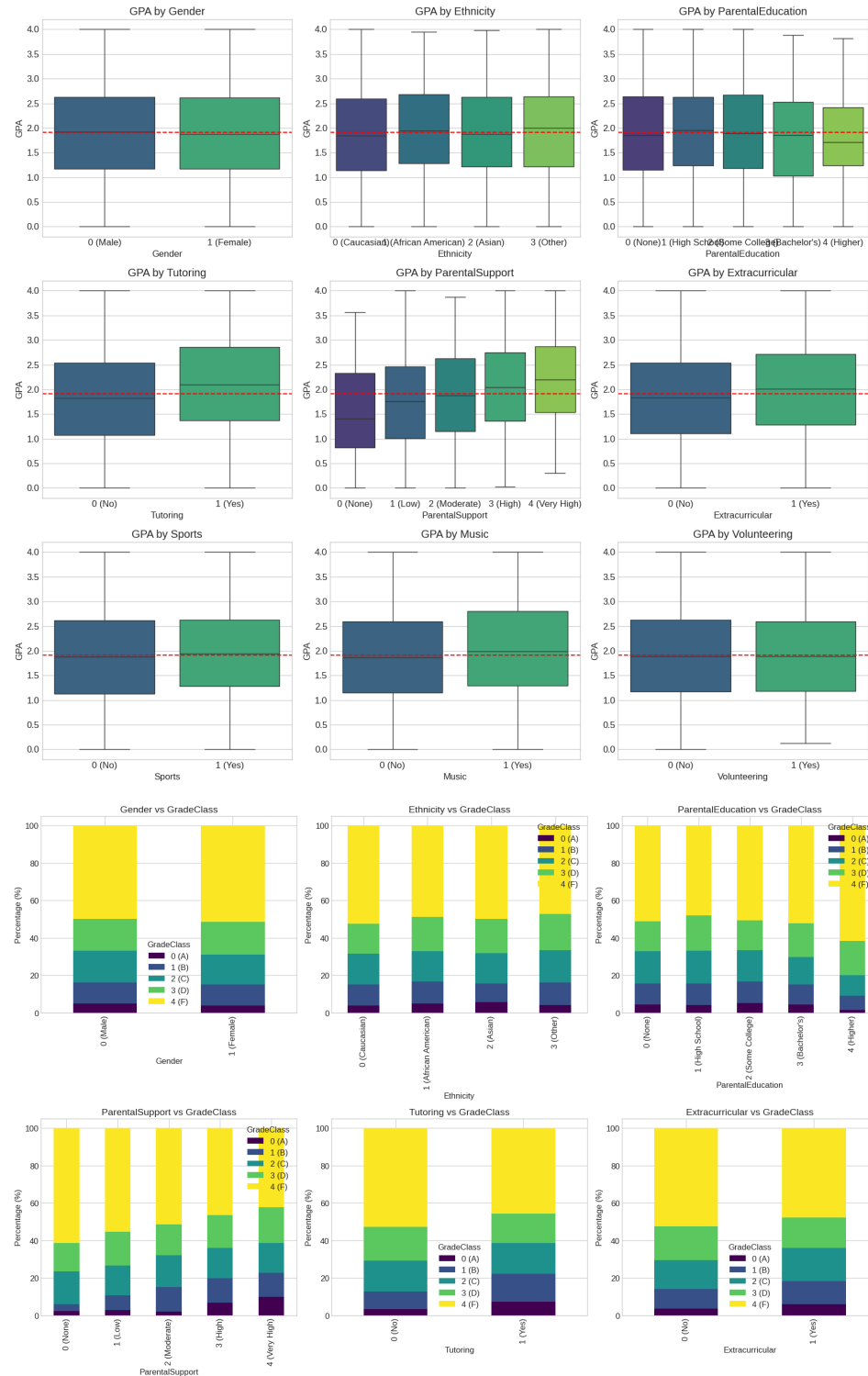


Visualisasi distribusi frekuensi untuk variabel kategorikal memperkuat temuan data sebelumnya. Terlihat bahwa mayoritas siswa adalah perempuan dan berasal dari etnis Kaukasasia. Sebagian besar orang tua memiliki tingkat pendidikan menengah ke atas, dengan dominasi pada kategori "Some College". Dukungan orang tua terhadap pendidikan anak cenderung berada di tingkat "Moderate" hingga "High". Namun, sebagian besar siswa tidak mengikuti les tambahan, tidak aktif dalam kegiatan ekstrakurikuler, olahraga, musik, maupun kegiatan sukarela, yang menunjukkan keterbatasan partisipasi dalam aktivitas di luar sekolah. Distribusi nilai memperlihatkan bahwa lebih dari separuh siswa memperoleh nilai F, yang menunjukkan potensi masalah dalam performa akademik secara umum. Kombinasi rendahnya keterlibatan

dalam aktivitas pendukung pembelajaran dan hasil akademik yang buruk ini dapat menjadi fokus penting untuk intervensi pendidikan ke depan.

8. Hubungan Antara Variabel Kategorik dan Kinerja Akademik

Relationship Between Categorical Variables and GPA



Dari grafik yang ditampilkan, kita dapat menyimpulkan bahwa beberapa faktor memiliki pengaruh terhadap prestasi akademik (GradeClass atau GPA) siswa. Berdasarkan diagram batang bertumpuk, sebagian besar siswa berada

pada kategori nilai rendah (GradeClass 4/F), namun siswa dengan dukungan orang tua tinggi, yang mengikuti les (tutoring), dan yang terlibat dalam kegiatan ekstrakurikuler cenderung memiliki distribusi nilai yang lebih baik. Hal ini juga didukung oleh boxplot GPA, di mana siswa yang mendapat dukungan orang tua tinggi, mengikuti les, aktif dalam musik atau kegiatan sosial (volunteering), menunjukkan median GPA yang lebih tinggi dibandingkan kelompok lainnya. Sebaliknya, perbedaan berdasarkan gender, etnis, atau pendidikan orang tua tampak kurang signifikan terhadap GPA siswa. Dengan demikian, keterlibatan dan dukungan langsung dalam aktivitas belajar dan non-akademik tampak lebih berperan dalam peningkatan prestasi.

B. EKSPLORASI DATA DAN PRAPEMROSESAN

1. Menghilangkan Fitur yang Tidak Relevan

	Age	Gender	Ethnicity	ParentalEducation	StudyTimeWeekly	Absences	Tutoring	ParentalSupport	Extracurricular	Sports	Music	Volunteering	GradeClass
0	17	1	0	2	19.834	7	1	2	0	0	1	0	2.000
1	18	0	0	1	15.409	0	0	1	0	0	0	0	1.000
2	15	0	2	3	4.211	26	0	2	0	0	0	0	4.000
3	17	1	0	3	10.029	14	0	3	1	0	0	0	3.000
4	17	1	0	2	4.672	17	1	3	0	0	0	0	4.000

Dengan menggunakan perintah `df.drop(['StudentID', 'GPA'], axis=1)`, kita menghapus kolom 'StudentID' dan 'GPA' dari DataFrame `df`.

2. Memeriksa Missing Values

```
Age          0
Gender       0
Ethnicity    0
ParentalEducation  0
StudyTimeWeekly  0
Absences     0
Tutoring     0
ParentalSupport  0
Extracurricular  0
Sports       0
Music        0
Volunteering  0
GradeClass   0
dtype: int64
```

Output ini menunjukkan bahwa setiap kolom dalam DataFrame memiliki nilai 0 pada hasilnya, yang berarti tidak ada nilai yang hilang (missing values) pada setiap kolom.

3. Ekspansi Polinomial

```
df['ParentalEducation_StudyTime'] = df['ParentalEducation'] * df['StudyTimeWeekly']
df['StudyAbsenceRatio'] = df['StudyTimeWeekly'] / (df['Absences'] + 1)

numerical_features = df[['Age', 'StudyTimeWeekly', 'Absences', 'ParentalEducation_StudyTime', 'StudyAbsenceRatio']]

poly = PolynomialFeatures(degree=2, interaction_only=False, include_bias=False)
poly_features = poly.fit_transform(numerical_features)

poly_features_df = pd.DataFrame(poly_features, columns=poly.get_feature_names_out(numerical_features.columns))

df = pd.concat([df, poly_features_df], axis=1)
```

Kode ini menambahkan dua kolom baru ke DataFrame df, yaitu ParentalEducation_StudyTime dan StudyAbsenceRatio. Kemudian, kolom-kolom numerik dipilih untuk dilakukan transformasi polinomial derajat 2 menggunakan PolynomialFeatures. Hasil transformasi ini kemudian digabungkan dengan DataFrame df, menambah fitur baru untuk analisis lebih lanjut.

4. Menghapus Kolom Duplikat

```
## Remove duplicate columns if any
df = df.loc[:, ~df.columns.duplicated()]
```

Kode ini digunakan untuk menghapus kolom duplikat dalam DataFrame df. Dengan menggunakan df.loc[:, ~df.columns.duplicated()], kode ini memilih hanya kolom-kolom yang tidak terduplikasi, sehingga jika ada kolom yang memiliki nama yang sama, hanya satu yang akan dipertahankan.

5. Data Splitting

```
x = df.drop('GradeClass', axis=1)
y = df['GradeClass']

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.2, random_state = 42)
```

Kode ini mempersiapkan data untuk pelatihan model dengan membagi DataFrame df menjadi fitur (X) dan target (y). Kolom GradeClass dipisahkan sebagai target variabel y, sementara sisanya menjadi fitur yang disimpan dalam X. Kemudian, data dibagi menjadi data pelatihan dan pengujian menggunakan fungsi train_test_split, dengan 80% data digunakan untuk pelatihan (X_train, y_train) dan 20% sisanya untuk pengujian (X_test, y_test). Parameter random_state=42 memastikan pembagian data yang konsisten setiap kali kode dijalankan.

6. Scaling

```
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

Kode ini menormalisasi data fitur agar memiliki rata-rata 0 dan standar deviasi 1. Data pelatihan diubah menggunakan fit_transform, sementara data pengujian diubah dengan transform menggunakan skala yang sama, agar model lebih akurat dan tidak bias terhadap perbedaan skala.

C. IMPLEMENTASI MODEL

1. Logistic Regression dengan Hyperparameter Tuning

```
# Logistic Regression with hyperparameter tuning
print("\n📊 LOGISTIC REGRESSION")
print("="*80)
print("Performing hyperparameter tuning for Logistic Regression...")

# Define parameter grid
lr_param_grid = {
    'C': [0.01, 0.1, 1, 10, 100],
    'penalty': ['l1', 'l2'],
    'solver': ['liblinear'],
    'class_weight': [class_weights_dict, None]
}

# Create and train GridSearchCV
lr_grid = GridSearchCV(
    LogisticRegression(max_iter=1000),
    param_grid=lr_param_grid,
    cv=cv_strategy,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

lr_grid.fit(X_train_scaled, y_train)

# Get best model
lr_best = lr_grid.best_estimator_
print(f"Best parameters: {lr_grid.best_params_}")
print(f"Best cross-validation score: {lr_grid.best_score_:.4f}")

# Evaluate the model
lr_metrics = evaluate_model(lr_best, X_train_scaled, X_test_scaled, y_train, y_test, "Logistic Regression")
all_metrics.append(lr_metrics)
```

Pada algoritma logistic regression, grid parameter yang diuji adalah C, penalty apakah 'l1', atau 'l2', solver, dan class_weight apakah ditangani atau tidak. Kemudian parameter terbaik akan digunakan untuk membangun model. Kemudian model dilatih menggunakan data training. Lalu model terbaik akan dievaluasi menggunakan metrik evaluasi. Proses dari hyperparameter tuning dapat dilihat pada gambar di bawah.

```
📊 LOGISTIC REGRESSION
=====
Performing hyperparameter tuning for Logistic Regression...
Fitting 5 folds for each of 20 candidates, totalling 100 fits
Best parameters: {'C': 1, 'class_weight': {0: 4.501176470588235, 1: 1.739090909090909, 2: 1.250326797385621, 3: 1.1664634146341464, 4: 0.39281314168377823}, 'penalty': 'l1', 'solver': 'liblinear'}
Best cross-validation score: 0.5428
```

2. K-Nearest Neighbors dengan Hyperparameter Tuning

```
# K-Nearest Neighbors with hyperparameter tuning
print("\n🔍 K-NEAREST NEIGHBORS")
print("="*80)
print("Performing hyperparameter tuning for K-NN...")

# Define parameter grid
knn_param_grid = {
    'n_neighbors': [3, 5, 7, 9, 11, 13, 15],
    'weights': ['uniform', 'distance'],
    'p': [1, 2] # 1 for Manhattan, 2 for Euclidean
}
```

```

# Create and train GridSearchCV
knn_grid = GridSearchCV(
    ..., KNeighborsClassifier(),
    ..., param_grid=knn_param_grid,
    ..., cv=cv_strategy,
    ..., scoring='f1_macro',
    ..., n_jobs=-1,
    ..., verbose=1
)

knn_grid.fit(X_train_scaled, y_train)

# Get best model
knn_best = knn_grid.best_estimator_
print(f"Best parameters: {knn_grid.best_params_}")
print(f"Best cross-validation score: {knn_grid.best_score_:.4f}")

# Evaluate the model
knn_metrics = evaluate_model(knn_best, X_train_scaled, X_test_scaled, y_train, y_test, "K-Nearest Neighbors")
all_metrics.append(knn_metrics)

```

Langkah awalnya adalah mendefinisikan grid hyperparameter, mulai dari jumlah `n_neighbors`, `weights` apakah 'uniform' atau distance, lalu `p` apakah manhattan yang dilambangkan dengan 1 atau euclidian yang dilambangkan dengan 2. Proses dari hyperparameter tuning dapat dilihat pada gambar di bawah.

K-NEAREST NEIGHBORS

```

=====
Performing hyperparameter tuning for K-NN...
Fitting 5 folds for each of 28 candidates, totalling 140 fits
Best parameters: {'n_neighbors': 15, 'p': 1, 'weights': 'uniform'}
Best cross-validation score: 0.4901

```

Dari gambar di atas, terlihat bahwa best paramnya adalah `n_neighbors` dengan jumlah 15, `p` yaitu manhattan, dan pemberian bobot atau `weights` di lakukan dengan cara yang sama untuk tiap tetangga.

3. Naive Bayes

```

# Naive Bayes
print("\n  NAIVE BAYES")
print("="*80)
print("Training Gaussian Naive Bayes model...")

# For Naive Bayes, we'll use cross-validation to assess performance
nb = GaussianNB()

# Perform cross-validation
cv_scores = cross_val_score(nb, X_train_scaled, y_train, cv=cv_strategy, scoring='f1_macro')
print(f"Cross-validation scores: {cv_scores}")
print(f"Mean CV score: {cv_scores.mean():.4f}")
print(f"Standard deviation: {cv_scores.std():.4f}")

# Train the model on the full training set
nb.fit(X_train_scaled, y_train)

# Evaluate the model
nb_metrics = evaluate_model(nb, X_train_scaled, X_test_scaled, y_train, y_test, "Naive Bayes")
all_metrics.append(nb_metrics)

```

Jenis Naive Bayes yang digunakan adalah Gaussian Naive Bayes yang mengasumsikan bahwa fitur mengikuti distribusi normal. Setelah melakukan cross-validation, hasil cross-validation akan ditampilkan. Kemudian membangun model dan melakukan evaluasi model. Hasil cross-validation dapat dilihat di bawah ini.

🧠 NAIVE BAYES

```
=====
Training Gaussian Naive Bayes model...
Cross-validation scores: [0.46062867 0.41627193 0.53469367 0.42131102 0.43234535]
Mean CV score: 0.4531
Standard deviation: 0.0436
```

4. Decision Tree dengan Hyperparameter Tuning

```
# Decision Tree with hyperparameter tuning
print("\n 🌳 DECISION TREE")
print("="*80)
print("Performing hyperparameter tuning for Decision Tree...")

# Define parameter grid
dt_param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [None, 5, 10, 15, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'class_weight': [class_weights_dict, None]
}

# Create and train GridSearchCV
dt_grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid=dt_param_grid,
    cv=cv_strategy,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

dt_grid.fit(X_train_scaled, y_train)

# Get best model
dt_best = dt_grid.best_estimator_
print(f"Best parameters: {dt_grid.best_params_}")
print(f"Best cross-validation score: {dt_grid.best_score_:.4f}")

# Evaluate the model
dt_metrics = evaluate_model(dt_best, X_train_scaled, X_test_scaled, y_train, y_test, "Decision Tree")
all_metrics.append(dt_metrics)

# Visualize the Decision Tree (limiting depth for readability)
plt.figure(figsize=(20, 12))
plot_tree(dt_best,
           max_depth=3,
           filled=True,
           feature_names=X_train.columns,
           class_names=[f'Grade {i}' for i in range(len(np.unique(y_test)))],
           rounded=True)
plt.title("Decision Tree Visualization (Limited to Depth 3)")
plt.tight_layout()
plt.show()
```

Kami melakukan pencarian hyperparameter terbaik menggunakan GridSearchCV. Parameter yang diuji dalam tuning mulai dari criterion apakah 'gini' atau 'entropy', max_depth, min_samples_split, min_samples_leaf, dan class weight apakah digunakan atau tidak. Kemudian menggunakan best parameter untuk menghasilkan model terbaik. Model kemudian dievaluasi dan divisualisasikan hingga kedalaman 3 untuk memudahkan interpretasi

D. EVALUASI MODEL

1. Persiapan evaluasi model klasifikasi

```
# Create a dictionary to store evaluation metrics for all models
all_metrics = []

# Function to evaluate and visualize model performance
def evaluate_model(model, X_train_scaled, X_test_scaled, y_train, y_test, model_name):
    """
    Evaluates model performance and creates visualizations.

    Args:
        model: Trained classifier model
        X_train_scaled: Scaled training features
        X_test_scaled: Scaled testing features
        y_train: Training target variable
        y_test: Testing target variable
        model_name: Name of the model for display purposes

    Returns:
        metrics_dict: Dictionary containing evaluation metrics
    """
    # Make predictions
    y_pred = model.predict(X_test_scaled)
    y_pred_proba = None

    # Check if the model supports probability predictions
    if hasattr(model, "predict_proba"):
        y_pred_proba = model.predict_proba(X_test_scaled)

    # Calculate metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision_macro = precision_score(y_test, y_pred, average='macro')
    recall_macro = recall_score(y_test, y_pred, average='macro')
    f1_macro = f1_score(y_test, y_pred, average='macro')

    # Store metrics in dictionary
    metrics_dict = {
        'Model': model_name,
        'Accuracy': accuracy,
        'Precision (macro)': precision_macro,
        'Recall (macro)': recall_macro,
        'F1 Score (macro)': f1_macro
    }

    # Print metrics
    print(f"\n{'-'*20} {model_name} Evaluation {'-'*20}")
    print(f"Accuracy: {accuracy:.4f}")
    print(f"Precision (macro): {precision_macro:.4f}")
    print(f"Recall (macro): {recall_macro:.4f}")
    print(f"F1 Score (macro): {f1_macro:.4f}")
```

```

# Print classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=[f'Grade {i}' for i in range(len(np.unique(y_test))]))

# Create confusion matrix
plt.figure(figsize=(10, 8))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=[f'Grade {i}' for i in range(len(np.unique(y_test))]),
            yticklabels=[f'Grade {i}' for i in range(len(np.unique(y_test))]))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title(f'Confusion Matrix - {model_name}')
plt.tight_layout()
plt.show()

```

```

# If model supports probabilities, plot ROC curves
if y_pred_proba is not None:
    # For multiclass, use one-vs-rest approach
    n_classes = len(np.unique(y_test))

    if n_classes > 2:
        # Binarize the output for multi-class ROC
        y_test_bin = label_binarize(y_test, classes=np.unique(y_test))

        # Plot ROC curves
        plt.figure(figsize=(12, 8))

        for i in range(n_classes):
            fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_pred_proba[:, i])
            roc_auc = auc(fpr, tpr)
            plt.plot(fpr, tpr, lw=2,
                    label=f'ROC curve for Grade {i} (AUC = {roc_auc:.2f})')

        plt.plot([0, 1], [0, 1], 'k--', lw=2)
        plt.xlim([0.0, 1.0])
        plt.ylim([0.0, 1.05])
        plt.xlabel('False Positive Rate')
        plt.ylabel('True Positive Rate')
        plt.title(f'ROC Curves - {model_name}')
        plt.legend(loc="lower right")
        plt.show()

    else: # Binary classification
        fpr, tpr, _ = roc_curve(y_test, y_pred_proba[:, 1])
        roc_auc = auc(fpr, tpr)

        plt.figure(figsize=(10, 8))
        plt.plot(fpr, tpr, color='darkorange', lw=2,
                label=f'ROC curve (AUC = {roc_auc:.2f})')
        plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
        plt.xlim([0.0, 1.0])
        plt.ylim([0.0, 1.05])
        plt.xlabel('False Positive Rate')
        plt.ylabel('True Positive Rate')
        plt.title(f'ROC Curve - {model_name}')
        plt.legend(loc="lower right")
        plt.show()

return metrics_dict

```



```

# Computing class weights for handling imbalanced data
print("Computing class weights to handle class imbalance...")
class_counts = np.bincount(y_train.astype(int))
total_samples = len(y_train)

# Calculate class frequencies
class_frequencies = class_counts / total_samples
print(f"Class distribution: {class_frequencies}")

# Apply class weights (assuming imbalance exists)
class_weights = class_weight.compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train),
    y=y_train
)

# Convert to dictionary
class_weights_dict = {i: weight for i, weight in enumerate(class_weights)}
print(f"Class weights: {class_weights_dict}")

# Create cross-validation strategy (to be used for all models)
cv_strategy = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

```

Sebelum mengevaluasi model klasifikasi, kami menyiapkan beberapa komponen penting seperti metrik evaluasi (akurasi, presisi, recall, F1), penanganan ketidakseimbangan kelas dengan class weighting, serta validasi silang menggunakan Stratified K-Fold agar hasil evaluasi baik.

2. Evaluasi algoritma Logistic Regression

```

----- Logistic Regression Evaluation -----
Accuracy: 0.6785
Precision (macro): 0.5035
Recall (macro): 0.4954
F1 Score (macro): 0.4988

Classification Report:

```

	precision	recall	f1-score	support
Grade 0	0.16	0.14	0.15	22
Grade 1	0.47	0.41	0.43	49
Grade 2	0.53	0.52	0.52	85
Grade 3	0.48	0.51	0.50	86
Grade 4	0.88	0.90	0.89	237
accuracy			0.68	479
macro avg	0.50	0.50	0.50	479
weighted avg	0.67	0.68	0.67	479

Dari gambar diatas, dapat dilihat bahwa, dengan parameter terbaik, akurasi yang di dapat hanya 0,6785 atau sekitar 68% benar. Macro precision/Recall/F1 hanya 0,5 yang menunjukkan bahwa performa model tidak seimbang antar kelas, nilai ini memberi indikasi kelas minoritas diprediksi buruk.

3. Evaluasi algoritma K-Nearest Neighbors

```
----- K-Nearest Neighbors Evaluation -----
Accuracy: 0.6263
Precision (macro): 0.4981
Recall (macro): 0.4192
F1 Score (macro): 0.4348

Classification Report:
              precision    recall  f1-score   support

   Grade 0       0.50      0.14      0.21        22
   Grade 1       0.42      0.31      0.35        49
   Grade 2       0.41      0.36      0.39        85
   Grade 3       0.37      0.36      0.36        86
   Grade 4       0.79      0.93      0.85       237

 accuracy                0.63       479
 macro avg              0.50      0.42      0.43       479
weighted avg              0.60      0.63      0.60       479
```

Akurasi yang didapat adalah 62% prediksi benar secara keseluruhan. Grade 4 terlihat mendominasi karena performanya yang tinggi dibandingkan dengan kelas lainnya, terutama pada recallnya yang mencapai 0,93. Untuk grade 0 sangat rendah recallnya yaitu hanya 0,14 yang artinya mayoritas data grade 0 diklasifikasikan sebagai kelas lain.

4. Evaluasi algoritma Naive Bayes

```
----- Naive Bayes Evaluation -----
Accuracy: 0.6326
Precision (macro): 0.5280
Recall (macro): 0.4588
F1 Score (macro): 0.4518

Classification Report:
              precision    recall  f1-score   support

   Grade 0       0.38      0.14      0.20        22
   Grade 1       0.53      0.20      0.29        49
   Grade 2       0.43      0.33      0.37        85
   Grade 3       0.40      0.81      0.53        86
   Grade 4       0.91      0.81      0.86       237

 accuracy                0.63       479
 macro avg              0.53      0.46      0.45       479
weighted avg              0.67      0.63      0.63       479
```

Akurasi umum yang di dapat adalah 0,6326 atau sebanyak 63% data berhasil diprediksi dengan benar. Evaluasi makro menghitung metrik tanpa mempertimbangkan jumlah data di tiap kelas. Untuk precision (makro) didapat 0,5280 yang artinya model cukup baik menghindari false positive di semua kelas, tetapi tidak tinggi. Kemudian jika dilihat performa perkelasnya, grade 0 - grade 3 memiliki f1-score yang rendah, ini menunjukkan bahwa model sulit membedakan dan

mengenali kelas-kelas ini. Sedangkan grade 4 memiliki f1-score yang cukup baik, yaitu 0,86, yang artinya model sangat bagus dalam mengenali grade 4, tetapi ini menunjukkan ketimpangan data dan kemungkinan model bias ke grade 4. Karena hanya pada grade 4, support mencapai 200 lebih, sedangkan pada grade lainnya supportnya di bawah 100, bahkan grade 0 hanya memiliki support sejumlah 22..

5. Evaluasi algoritma Decision Tree

```

----- Decision Tree Evaluation -----
Accuracy: 0.6493
Precision (macro): 0.5306
Recall (macro): 0.4745
F1 Score (macro): 0.4812

Classification Report:

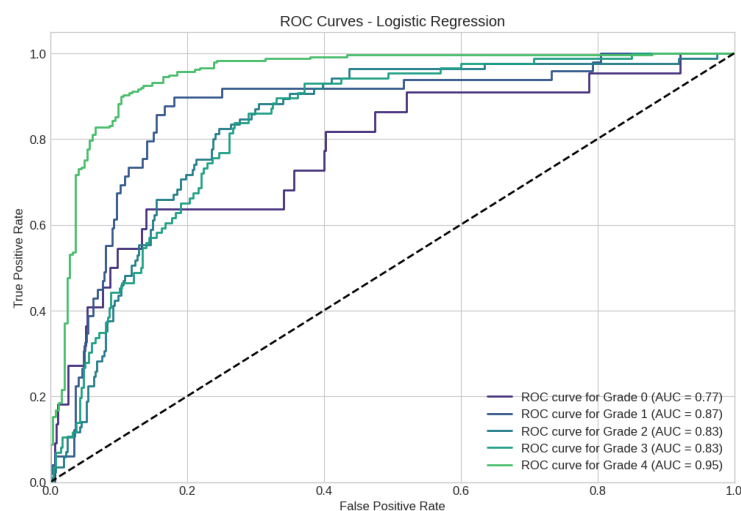
```

	precision	recall	f1-score	support
Grade 0	0.50	0.14	0.21	22
Grade 1	0.48	0.51	0.50	49
Grade 2	0.44	0.48	0.46	85
Grade 3	0.38	0.35	0.36	86
Grade 4	0.85	0.89	0.87	237
accuracy			0.65	479
macro avg	0.53	0.47	0.48	479
weighted avg	0.64	0.65	0.64	479

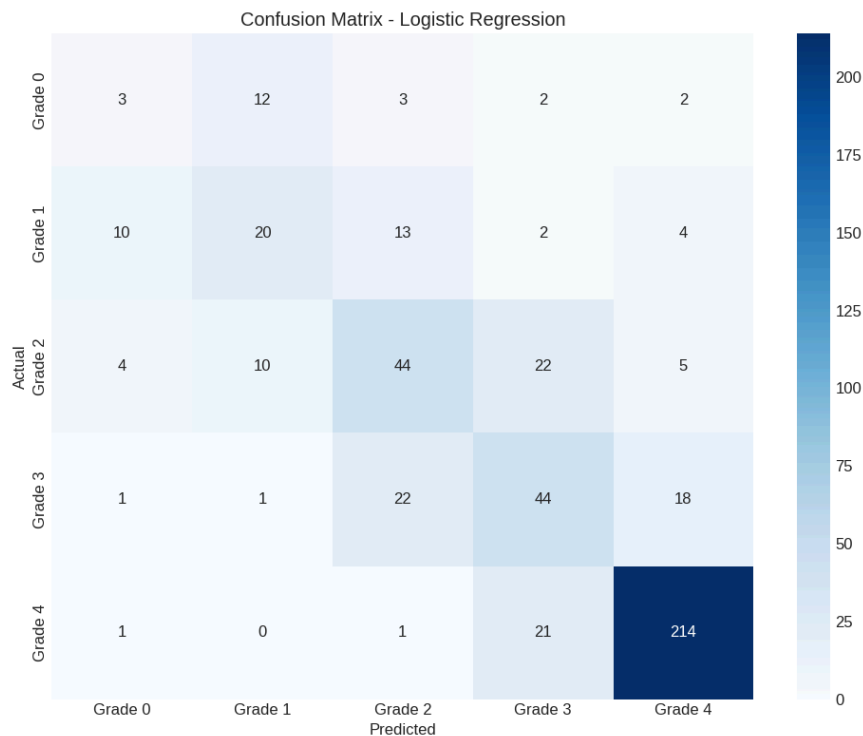
Algoritma decision tree menghasilkan sekitar 65% data yang berhasil diprediksi benar. Ini sedikit lebih baik dari pada naive bayes (63,26%). Nilai makro yang diperoleh memiliki keseimbangan yang lebih baik antar kelas dibandingkan naive bayes, meskipun tidak signifikan. Jika melihat dari performa perkelas, grade 4 sangat dominan dengan performa yang lebih bagus daripada kelas lainnya.

E. ANALISIS HASIL

1. Analisa Hasil Logistic Regression



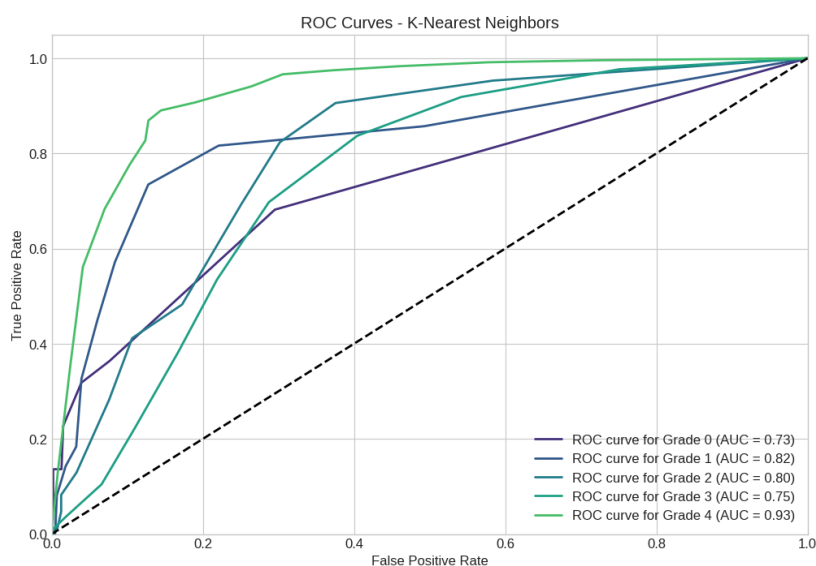
Gambar 1



Gambar 2

Gambar pertama menampilkan kurva ROC untuk model regresi logistik pada masing-masing kelas Grade 0 hingga Grade 4. Nilai AUC tertinggi terdapat pada Grade 4 (0.95), menunjukkan kinerja model sangat baik dalam mengklasifikasikan kelas ini, sementara Grade 0 memiliki AUC terendah (0.77). Gambar kedua adalah confusion matrix yang memperlihatkan prediksi model dibandingkan dengan label aktual. Model paling akurat dalam memprediksi Grade 4 dengan 214 prediksi benar, sedangkan untuk kelas lain seperti Grade 0 dan Grade 1, model masih mengalami banyak kesalahan klasifikasi, menunjukkan perlu adanya perbaikan untuk meningkatkan presisi pada kelas-kelas tersebut.

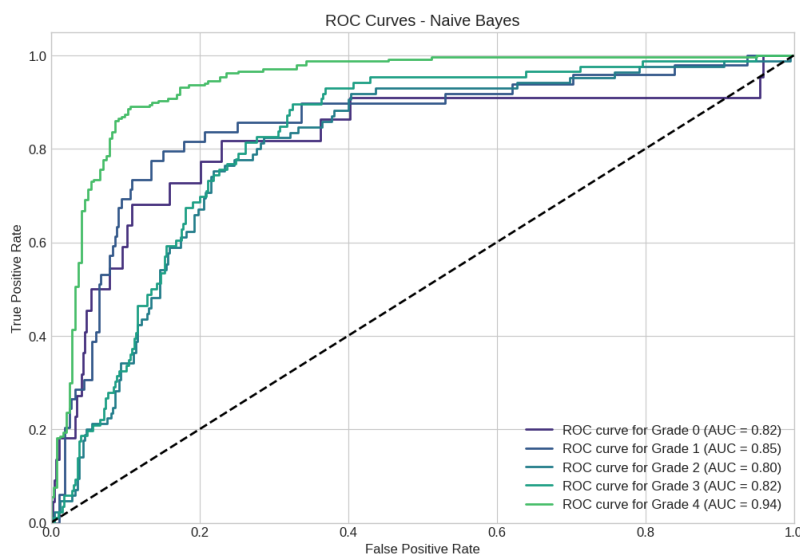
2. Analisa Hasil K-Nearest Neighbors

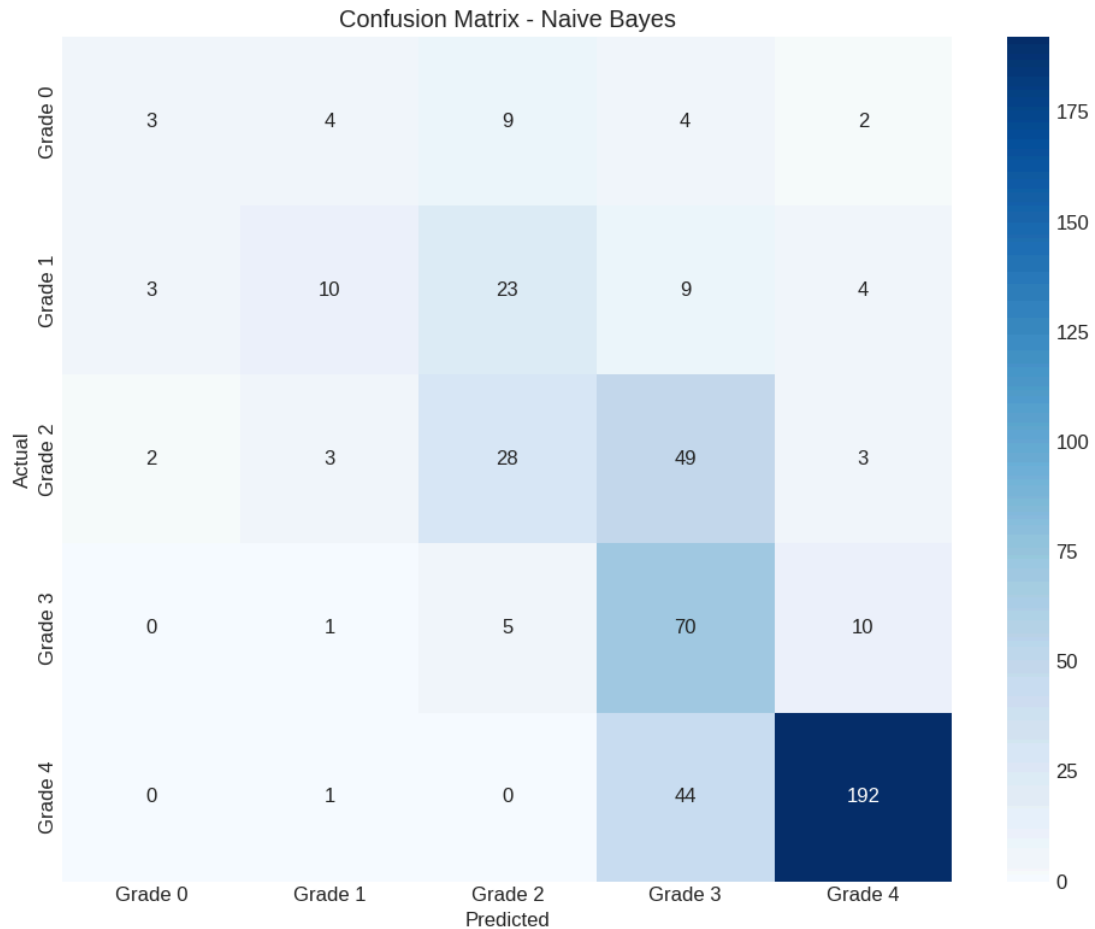




Pada grafik ROC, terlihat bahwa model memiliki performa terbaik pada kelas Grade 4 dengan nilai AUC tertinggi sebesar 0.93, diikuti oleh Grade 1 (0.82), Grade 2 (0.80), Grade 3 (0.75), dan Grade 0 (0.73). Hal ini menunjukkan bahwa model paling mampu membedakan Grade 4 dibandingkan kelas lainnya. Sementara itu, confusion matrix menunjukkan bahwa kelas Grade 4 memiliki jumlah prediksi benar (220) yang sangat tinggi, namun beberapa kelas lain seperti Grade 2 dan Grade 3 mengalami banyak kesalahan klasifikasi, terutama antar Grade yang berdekatan. Misalnya, Grade 2 sering salah diklasifikasikan sebagai Grade 3, dan sebaliknya. Secara keseluruhan, model menunjukkan performa yang cukup baik namun masih perlu peningkatan pada klasifikasi antar kelas yang serupa.

3. Analisa Hasil Naive Bayes

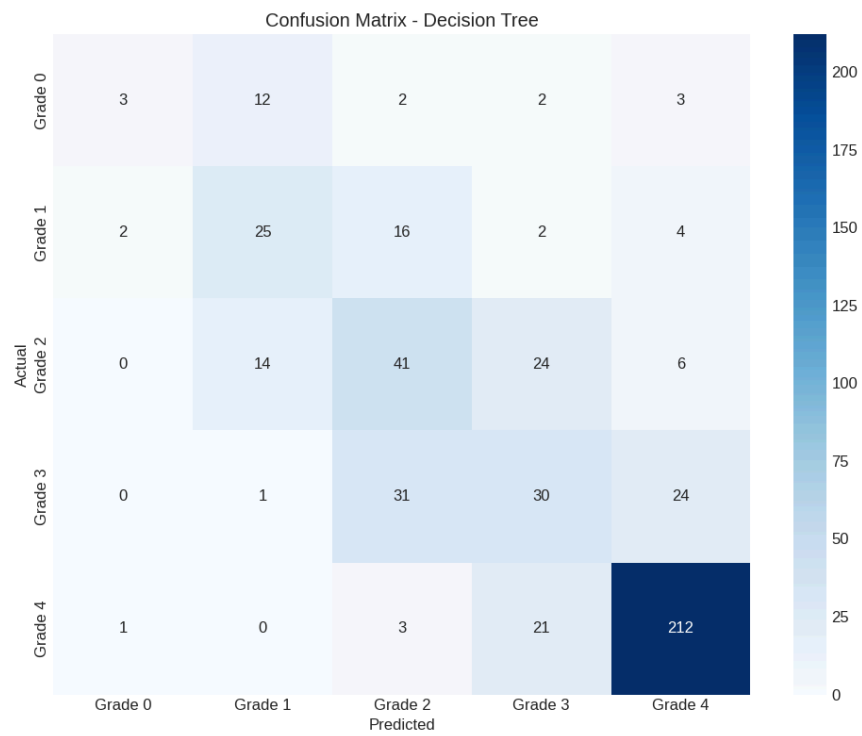
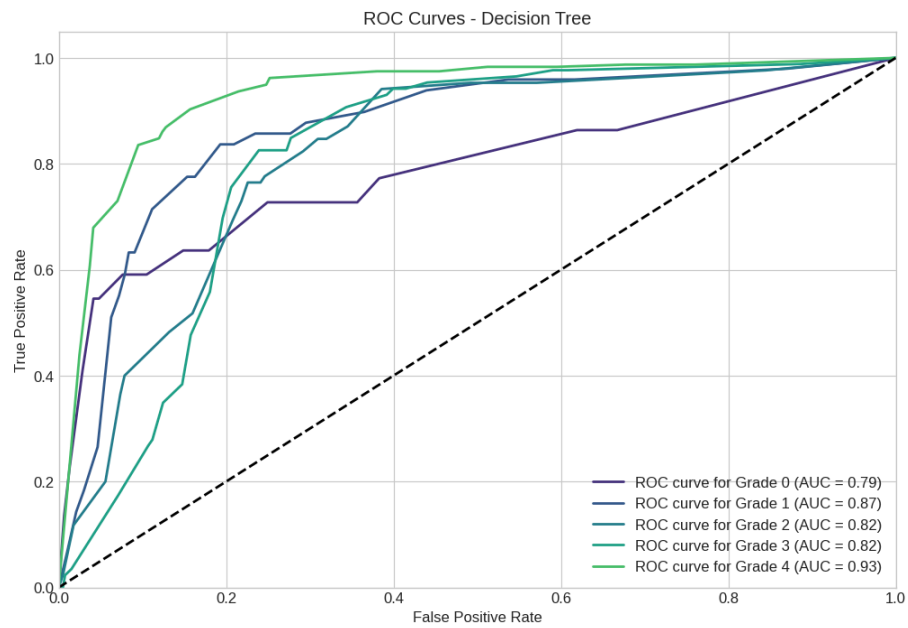


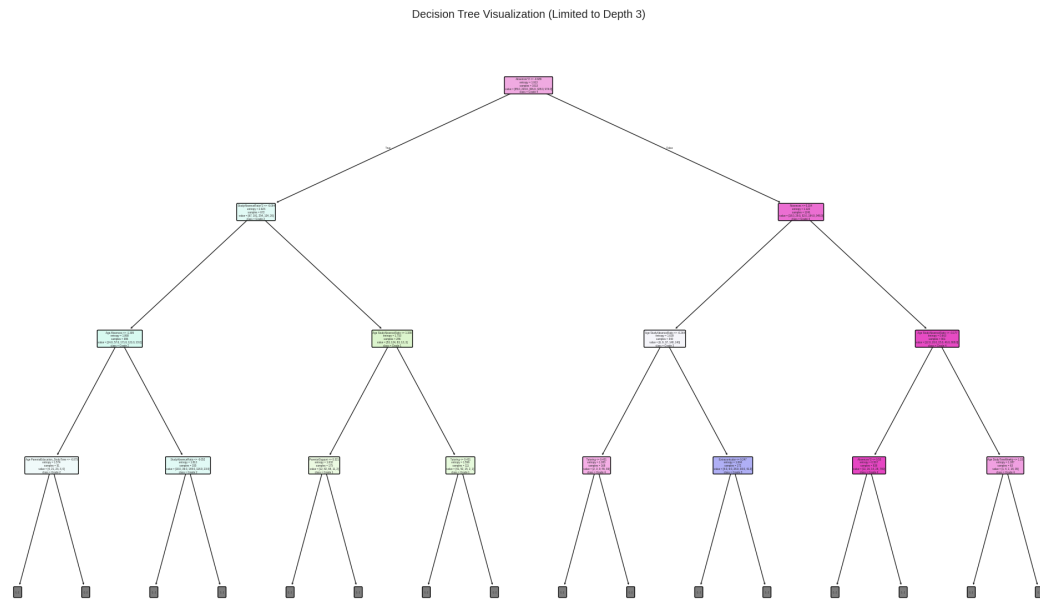


Visualisasi hasil klasifikasi dengan metode Naive Bayes menunjukkan performa yang cukup baik berdasarkan ROC Curve dan confusion matrix. Grafik ROC untuk masing-masing kelas (Grade 0 hingga Grade 4) memperlihatkan nilai AUC (Area Under Curve) yang cukup tinggi, dengan Grade 4 memiliki AUC tertinggi sebesar 0.94, menandakan kemampuan model dalam membedakan kelas ini sangat baik. Nilai AUC untuk kelas lainnya juga tergolong baik, yaitu antara 0.80 hingga 0.85.

Sementara itu, confusion matrix memberikan gambaran detail mengenai distribusi prediksi model. Terlihat bahwa sebagian besar kelas Grade 4 berhasil diprediksi dengan akurat (192 prediksi benar dari total 237), meskipun masih ada sedikit kesalahan klasifikasi, terutama dari kelas-kelas yang berdekatan seperti Grade 2 dan Grade 3. Kesalahan klasifikasi paling sering terjadi antar kelas yang nilainya berdekatan, seperti Grade 2 yang sering salah diklasifikasikan sebagai Grade 3, dan sebaliknya. Hal ini menunjukkan bahwa meskipun model cukup andal, terdapat tantangan dalam membedakan kelas-kelas yang memiliki karakteristik mirip.

5. Analisa Hasil Decision Tree

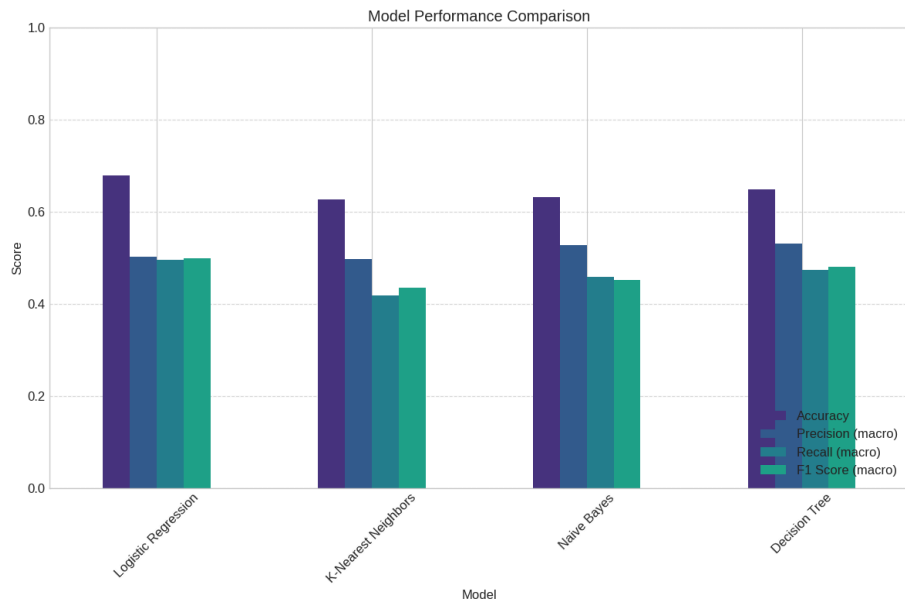




Pada kurva ROC, nilai AUC (Area Under Curve) untuk setiap kelas menunjukkan performa yang cukup baik, dengan nilai tertinggi pada Grade 4 (AUC = 0.93) dan terendah pada Grade 0 (AUC = 0.79), yang mengindikasikan kemampuan model membedakan antar kelas cukup baik, terutama pada kelas mayoritas. Confusion matrix menunjukkan bahwa model paling akurat dalam mengklasifikasikan Grade 4 dengan 212 prediksi benar, sementara kelas seperti Grade 0 dan Grade 1 mengalami lebih banyak kesalahan klasifikasi, kemungkinan karena jumlah datanya lebih sedikit atau fitur yang kurang diskriminatif. Visualisasi pohon keputusan (Decision Tree) memperlihatkan struktur aturan yang digunakan model dalam pengambilan keputusan, memberikan interpretasi yang jelas terhadap fitur-fitur penting dalam klasifikasi. Secara keseluruhan, model ini cukup andal namun masih bisa ditingkatkan, khususnya dalam membedakan kelas-kelas minoritas.

6. Analisis Visual Perbandingan Kinerja Model

Model Performance Comparison:				
	Accuracy	Precision (macro)	Recall (macro)	F1 Score (macro)
Model				
Logistic Regression	0.678	0.503	0.495	0.499
K-Nearest Neighbors	0.626	0.498	0.419	0.435
Naive Bayes	0.633	0.528	0.459	0.452
Decision Tree	0.649	0.531	0.474	0.481



Gambar di atas menunjukkan perbandingan performa empat model klasifikasi berdasarkan metrik Accuracy, Precision (macro), Recall (macro), dan F1 Score (macro). Dari hasil tersebut, terlihat bahwa Logistic Regression memiliki akurasi tertinggi (0.678) dibandingkan dengan model lainnya. Namun, nilai F1 Score-nya hanya 0.499, yang menunjukkan adanya ketidakseimbangan dalam memprediksi semua kelas secara adil. Decision Tree, di sisi lain, tampil paling seimbang dengan F1 Score (0.481), Precision (0.531), dan Recall (0.474) yang relatif stabil, menjadikannya model yang baik untuk kasus multikelas. Naive Bayes, meskipun memiliki Precision yang cukup tinggi (0.528), menunjukkan nilai Recall dan F1 Score yang lebih rendah dibandingkan dengan Logistic Regression dan Decision Tree. K-Nearest Neighbors (KNN) menunjukkan performa terendah, terutama pada metrik Recall dan F1 Score, yang mengindikasikan kesulitan model ini dalam menangani distribusi antar kelas pada dataset ini. Secara keseluruhan, Decision Tree dapat dianggap sebagai model terbaik berdasarkan keseimbangan antar metrik makro.

Kesimpulan

Berdasarkan hasil perbandingan performa empat model klasifikasi, dapat disimpulkan bahwa Decision Tree merupakan model yang paling seimbang dalam hal metrik makro, dengan Precision, Recall, dan F1 Score yang relatif stabil. Meskipun Logistic Regression memiliki akurasi tertinggi, nilai F1 Score-nya yang rendah menunjukkan bahwa model ini kurang mampu memprediksi semua kelas secara adil. Naive Bayes memiliki Precision yang baik, namun recall dan F1 Score-nya lebih rendah, sehingga kurang efektif dalam menangani distribusi antar kelas. K-Nearest Neighbors (KNN) menunjukkan performa yang paling buruk, terutama dalam hal recall dan F1 Score, yang mengindikasikan kesulitan dalam menangani distribusi kelas yang tidak seimbang. Secara keseluruhan, Decision Tree lebih direkomendasikan sebagai model yang lebih baik untuk kasus klasifikasi multikelas ini.